Automated Grant Intelligence System

Technical Implementation Specification

For Tech-Focused NGOs in Kenya

Version 1.0

February 2026

Table of Contents

1. Executive Summary

This document provides a comprehensive technical specification for building an automated grant intelligence and opportunity aggregation system optimized for technology-focused non-governmental organizations in Kenya, specifically tailored for organizations like Tech On Hand.

1.1 System Overview

The system automates the discovery, evaluation, and prioritization of international grant opportunities through a multi-stage semantic matching pipeline. It replaces manual grant searching with an intelligent aggregation system that surfaces only the most relevant opportunities to grant writers.

1.2 Key Design Decisions

Aspect	Decision	Rationale
Deployment	VPS (DigitalOcean/AWS)	Full control, persistent services, cost-effective
Budget	$40-50/month	Open-source stack, local models, no API costs
Matching	Multi-stage ranking	Prioritizes accuracy over speed for 1-2 grant writers
Sources (v1)	DevelopmentAid, FundsforNGOs, Terra Viva	International focus, RSS feeds available
Database	PostgreSQL + pgvector	Free, robust, vector search capable
Embeddings	sentence-transformers (local)	No API costs, high quality
Delivery	Daily email digest	Low volume, high relevance for small team

1.3 Success Metrics

- Precision: >80% of surfaced grants are genuinely relevant
- Time savings: 15+ hours/week reduction in manual searching
- Coverage: Monitor 3+ major international sources continuously
- Latency: New opportunities surfaced within 24 hours
- Uptime: >99% system availability

2. System Architecture

2.1 High-Level Architecture

The system follows a modular pipeline architecture with clear separation of concerns:

Ingestion Layer → Processing Layer → Matching Layer → Delivery Layer

2.1.1 Component Overview

- Ingestion Layer: RSS monitors and web scrapers collect raw grant data
- Processing Layer: Normalizes, deduplicates, and enriches grant records
- Matching Layer: Three-stage semantic pipeline (retrieval → rerank → validation)
- Delivery Layer: Generates and sends daily digests via email
- Admin Interface: FastAPI dashboard for monitoring and configuration

2.2 Data Flow

```
1. RSS Feed Poller (every 4 hours)
   ↓
2. Raw Grant Extraction
   ↓
3. Normalization & Validation (Pydantic models)
   ↓
4. Deduplication (URL hash → Title → Semantic)
   ↓
5. Vector Embedding Generation (MiniLM)
   ↓
6. Stage 1: Fast Retrieval (cosine similarity > 0.5)
   ↓
7. Stage 2: Precision Reranking (cross-encoder)
   ↓
8. Stage 3: LLM Validation (eligibility check)
   ↓
9. Daily Digest Generation (top 5-10 opportunities)
   ↓
10. Email Delivery (8 AM EAT)
```

3. Technology Stack

3.1 Core Technologies

Component	Technology	Version	Justification
Runtime	Python	3.11+	Mature ML/NLP ecosystem
Database	PostgreSQL	15+	ACID compliance, pgvector support
Vector Store	pgvector	0.5.1+	Native PostgreSQL extension, no extra service
Cache	Redis	7.0+	Fast rate limiting and deduplication
Task Queue	APScheduler	3.10+	Lightweight cron alternative
Web Framework	FastAPI	0.109+	Admin dashboard and API
RSS Parser	feedparser	6.0+	Robust RSS/Atom support
HTTP Client	httpx	0.26+	Async requests with retry logic
Web Scraping	Scrapy	2.11+	High-performance static scraping
Dynamic Scraping	Playwright	1.41+	JavaScript-rendered pages
Embeddings	sentence-transformers	2.3+	Local semantic search
LLM Runtime	Ollama	0.1.20+	Local LLM inference (Llama 3.1 8B)

3.2 ML Models

Model	Purpose	Size	Performance
all-MiniLM-L6-v2	Stage 1: Fast retrieval	23M params / 80MB	~5ms per query

cross-encoder/ms-marco-MiniLM-L6-v2	Stage 2: Reranking	110M params	~50ms per pair
Llama 3.1 8B (quantized)	Stage 3: Eligibility validation	8B params / 4.7GB	~2s per validation

3.3 Infrastructure Requirements

Resource	Minimum	Recommended
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB
Storage	20 GB SSD	40 GB SSD
Monthly Cost	$24/month	$48/month
Provider	DigitalOcean Droplet	AWS t3.medium

4. Database Schema

4.1 Core Tables

4.1.1 grants

```
CREATE TABLE grants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source_id VARCHAR(255) UNIQUE NOT NULL,
  url TEXT UNIQUE NOT NULL,
  url_hash VARCHAR(64) NOT NULL,
  title TEXT NOT NULL,
  description TEXT,
  funder_name VARCHAR(255),
  funder_type VARCHAR(50), -- foundation, government, corporate
  funding_amount_min DECIMAL(12,2),
  funding_amount_max DECIMAL(12,2),
  currency VARCHAR(3) DEFAULT 'USD',
  deadline TIMESTAMP,
  geographic_eligibility TEXT[], -- Array of countries/regions
  cause_tags TEXT[], -- ['Digital Literacy', 'Hardware', 'Education']
  embedding vector(384), -- MiniLM embedding
  discovered_at TIMESTAMP DEFAULT NOW(),
  last_updated TIMESTAMP DEFAULT NOW(),
  is_active BOOLEAN DEFAULT TRUE
);


CREATE INDEX idx_grants_url_hash ON grants(url_hash);
CREATE INDEX idx_grants_deadline ON grants(deadline) WHERE is_active = TRUE;
CREATE INDEX idx_grants_embedding ON grants USING ivfflat (embedding
vector_cosine_ops);
```

4.1.2 grant_matches

```
CREATE TABLE grant_matches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  grant_id UUID REFERENCES grants(id) ON DELETE CASCADE,
  retrieval_score DECIMAL(4,3), -- Stage 1 cosine similarity
  rerank_score DECIMAL(4,3), -- Stage 2 cross-encoder score
  llm_score DECIMAL(4,3), -- Stage 3 LLM validation (0-1)
```

```
  llm_reasoning TEXT, -- Why it matches/doesn't
  final_score DECIMAL(4,3) GENERATED ALWAYS AS (
    (retrieval_score * 0.2) + (rerank_score * 0.3) + (llm_score * 0.5)
  ) STORED,
  matched_at TIMESTAMP DEFAULT NOW(),
  sent_in_digest BOOLEAN DEFAULT FALSE
);

CREATE INDEX idx_matches_final_score ON grant_matches(final_score DESC);
```

4.1.3 sources

```
CREATE TABLE sources (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) UNIQUE NOT NULL,
  type VARCHAR(20) NOT NULL, -- 'rss', 'scraper'
  url TEXT NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  check_interval_hours INTEGER DEFAULT 4,
  last_checked TIMESTAMP,
  last_success TIMESTAMP,
  consecutive_failures INTEGER DEFAULT 0
);
```

4.1.4 scraping_logs

```
CREATE TABLE scraping_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source_id UUID REFERENCES sources(id),
  started_at TIMESTAMP DEFAULT NOW(),
  completed_at TIMESTAMP,
  status VARCHAR(20), -- 'success', 'partial', 'failed'
  grants_found INTEGER,
  grants_new INTEGER,
  error_message TEXT
);
```

5. Ingestion Layer Implementation

5.1 RSS Feed Monitoring

RSS feeds are the foundation of the system due to their reliability and ethical compliance.

5.1.1 Target RSS Feeds

Source	URL	Update Frequency
DevelopmentAid Grants	https://www.developmentaid.org/rss/grants	Daily
FundsforNGOs Africa	https://www2.fundsforngos.org/category/africa/feed/	2-3x daily
Terra Viva Grants	https://terra1830.rssing.com/index.php	Daily

5.1.2 RSS Poller Implementation

```
# rss_poller.py
import feedparser
import hashlib
from datetime import datetime

class RSSPoller:
    def __init__(self, db_conn, redis_client):
        self.db = db_conn
        self.redis = redis_client

    async def poll_source(self, source):
        feed = feedparser.parse(source.url)
        new_grants = []

        for entry in feed.entries:
            url_hash = hashlib.sha256(entry.link.encode()).hexdigest()

            # Check if already processed (Redis cache)
            if self.redis.exists(f'processed:{url_hash}'):
                continue
```

```
            grant_data = {
                'source_id': f'{source.name}:{entry.id}',
                'url': entry.link,
                'url_hash': url_hash,
                'title': entry.title,
                'description': entry.get('summary', ''),
                'discovered_at': datetime.now()
            }

            new_grants.append(grant_data)
            self.redis.setex(f'processed:{url_hash}', 86400, '1')

        return new_grants
```

5.2 Web Scraping Layer

For sources without RSS feeds, implement targeted scrapers with ethical compliance.

5.2.1 Ethical Scraping Guidelines

- Always respect robots.txt directives
- Use transparent User-Agent identifying organization
- Implement rate limiting (2-5 seconds between requests)
- Scrape only during off-peak hours (2 AM - 6 AM EAT)
- Cache aggressively to minimize repeat requests
- Monitor for 429/403 responses and back off exponentially

5.2.2 Scraper Example (Future Phase)

```
# scrapers/nrf_kenya.py
import scrapy
from scrapy.crawler import CrawlerProcess


class NRFKenyaSpider(scrapy.Spider):
    name = 'nrf_kenya'
    allowed_domains = ['nrf.go.ke']
    start_urls = ['https://www.nrf.go.ke/grants']

    custom_settings = {
        'DOWNLOAD_DELAY': 3,
        'USER_AGENT': 'TechOnHand-GrantBot/1.0 (+grants@techonhand.org)'
    }
```

```
def parse(self, response):
    for grant in response.css('div.grant-item'):
        yield {
            'title': grant.css('h3::text').get(),
            'url': grant.css('a::attr(href)').get(),
            'deadline': grant.css('span.deadline::text').get()
        }
```

6. Processing Layer

6.1 Data Normalization

All raw data is validated and normalized using Pydantic models to ensure consistency.

6.1.1 Pydantic Schema

```
# models.py
from pydantic import BaseModel, Field, HttpUrl
from typing import Optional, List
from datetime import datetime


class GrantRecord(BaseModel):
    source_id: str
    url: HttpUrl
    title: str = Field(..., min_length=5, max_length=500)
    description: Optional[str] = None
    funder_name: Optional[str] = None
    funder_type: Optional[str] = None
    funding_amount_min: Optional[float] = None
    funding_amount_max: Optional[float] = None
    currency: str = 'USD'
    deadline: Optional[datetime] = None
    geographic_eligibility: List[str] = []
    cause_tags: List[str] = []
```

6.2 Deduplication Strategy

Three-tier deduplication prevents redundant alerts from multiple sources.

6.2.1 Tier 1: URL Hash

```
url_hash = hashlib.sha256(grant.url.encode('utf-8')).hexdigest()
if db.exists_url_hash(url_hash):
    return 'DUPLICATE_URL'
```

6.2.2 Tier 2: Funder + Title

```
composite_key = f"{grant.funder_name}::{grant.title}".lower()
if db.exists_composite_key(composite_key):
    return 'DUPLICATE_LOGICAL'
```

6.2.3 Tier 3: Semantic Similarity

```
# Only check new grants from last 30 days
recent_grants = db.get_recent_grants(days=30)
for existing in recent_grants:
    similarity = cosine_similarity(grant.embedding, existing.embedding)
    if similarity > 0.95:
        return f'DUPLICATE_SEMANTIC (score={similarity:.3f})'
```

6.3 Field Enrichment

6.3.1 Geographic Eligibility Extraction

```
# Use NLP to extract countries/regions from description
def extract_geographic_eligibility(description: str) -> List[str]:
    keywords = ['Kenya', 'East Africa', 'Sub-Saharan Africa', 'developing
countries']
    found = []
    for keyword in keywords:
        if keyword.lower() in description.lower():
            found.append(keyword)
    return found
```

6.3.2 Cause Tag Assignment (Keyword-Based)

```
CAUSE_KEYWORDS = {
    'Digital Literacy': ['digital skills', 'computer training', 'ICT
education'],
    'Hardware': ['computers', 'laptops', 'refurbished', 'equipment'],
    'Education': ['schools', 'students', 'teachers', 'curriculum'],
    'Youth Development': ['youth', 'young people', 'adolescents'],
    'Circular Economy': ['recycling', 'e-waste', 'sustainability']
}
```

7. Three-Stage Semantic Matching Pipeline

This is the core differentiator of the system. We prioritize accuracy over speed by using a multi-stage ranking approach that progressively filters and reranks candidates.

7.1 Organization Mission Embedding

The organization's mission is converted into a dense vector representation that serves as the reference point for all matching.

7.1.1 Mission Statement Example

```
MISSION = '''
Tech On Hand bridges the digital divide in Kenya by refurbishing donated computer
hardware and distributing it to schools and youth centers. We provide digital skills
training to young people, focusing on practical technology education that prepares them
for employment. Our work encompasses hardware lifecycle management, e-waste recycling,
and partnerships with educational institutions to expand technology access in
marginalized communities.
'''
```

7.1.2 Embedding Generation

```
from sentence_transformers import SentenceTransformer


model = SentenceTransformer('all-MiniLM-L6-v2')
mission_embedding = model.encode(MISSION, convert_to_tensor=False)
# Result: 384-dimensional vector stored in config
```

7.2 Stage 1: Fast Retrieval (Bi-Encoder)

Purpose: Quickly filter thousands of grants down to ~50 candidates using vector similarity.

7.2.1 Implementation

```
# Generate embedding for each new grant
grant_text = f"{grant.title}. {grant.description}"
grant_embedding = model.encode(grant_text)


# Store in PostgreSQL with pgvector
```

```
db.execute(
    'INSERT INTO grants (title, description, embedding) VALUES (%s, %s, %s)',
    (grant.title, grant.description, grant_embedding.tolist())
)

# Retrieve candidates (cosine similarity > 0.5)
candidates = db.execute('''
    SELECT id, title, 1 - (embedding <=> %s::vector) as similarity
    FROM grants
    WHERE is_active = TRUE
      AND 1 - (embedding <=> %s::vector) > 0.5
    ORDER BY similarity DESC
    LIMIT 50
''', (mission_embedding, mission_embedding))
```

7.2.2 Performance

- Speed: ~5ms per query with ivfflat index
- Recall: Retrieves 95%+ of truly relevant grants
- Filters: 1000 grants → ~50 candidates

7.3 Stage 2: Precision Reranking (Cross-Encoder)

Purpose: Rerank the top 50 candidates using a more sophisticated model that processes mission-grant pairs jointly.

7.3.1 Cross-Encoder vs Bi-Encoder

Bi-encoders (Stage 1) encode mission and grant separately, then compare vectors. Cross-encoders process both texts together, allowing for deeper semantic understanding but at higher computational cost.

7.3.2 Implementation

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L6-v2')

# Rerank top 50 candidates
pairs = [(MISSION, f"{c.title}. {c.description}") for c in candidates]
rerank_scores = reranker.predict(pairs)
```

```
# Sort by rerank score
ranked = sorted(zip(candidates, rerank_scores), key=lambda x: x[1],
reverse=True)
top_candidates = ranked[:20]  # Keep top 20 for LLM validation
```

7.3.3 Performance

- Speed: ~50ms per pair × 50 pairs = ~2.5 seconds
- Precision: Accurately distinguishes subtle relevance differences
- Filters: 50 candidates → ~20 high-quality matches

7.4 Stage 3: LLM Eligibility Validation

Purpose: Use a local LLM to perform complex reasoning about eligibility criteria, geographic restrictions, and organizational fit.

7.4.1 Why LLM Validation?

- Parse complex eligibility requirements (e.g., '501(c)(3) or equivalent')
- Reason about geographic restrictions ('East Africa' includes Kenya)
- Detect subtle disqualifiers ('new hardware only' vs 'refurbished hardware')
- Provide human-readable explanations for grant writers

7.4.2 Implementation with Ollama

```
import ollama


VALIDATION_PROMPT = '''
Organization Mission:
{mission}


Grant Opportunity:
Title: {title}
Funder: {funder}
Description: {description}
Geographic Eligibility: {geo_eligibility}


Question: On a scale of 0.0 to 1.0, how well does this grant match the
organization's
mission and eligibility criteria? Consider:
1. Does the cause align with the organization's work?
2. Is the organization geographically eligible?
3. Are there any disqualifying requirements (e.g., must be registered in US)?
```

```
Respond in JSON format:
{
  "score": 0.85,
  "reasoning": "Strong match. Grant supports digital literacy in East
Africa..."
}
'''


response = ollama.generate(
    model='llama3.1:8b',
    prompt=VALIDATION_PROMPT.format(
        mission=MISSION,
        title=grant.title,
        funder=grant.funder_name,
        description=grant.description,
        geo_eligibility=grant.geographic_eligibility
    )
)


result = json.loads(response['response'])
llm_score = result['score']
llm_reasoning = result['reasoning']
```

7.4.3 Performance

- Speed: ~2 seconds per validation × 20 candidates = ~40 seconds
- Accuracy: Catches nuanced disqualifiers humans would notice
- Output: Top 5-10 grants with confidence scores and explanations

7.5 Final Score Calculation

The final score is a weighted combination of all three stages:

```
final_score = (retrieval_score * 0.2) + (rerank_score * 0.3) + (llm_score *
0.5)
```

Rationale for weights:

- Retrieval (20%): Fast but less precise, used for filtering
- Rerank (30%): More accurate than retrieval, general relevance
- LLM (50%): Most sophisticated, catches eligibility nuances

8. Delivery Layer: Daily Digest System

8.1 Digest Generation Logic

Since there are only 1-2 grant writers, we optimize for high relevance over real-time delivery.

8.1.1 Digest Criteria

- Frequency: Daily at 8:00 AM EAT
- Content: Top 5-10 grants with final_score > 0.7
- Grouping: By deadline urgency (< 7 days, 7-30 days, > 30 days)
- Format: HTML email with actionable summaries

8.1.2 Email Template Structure

```
Subject: Daily Grant Digest - {count} High-Quality Opportunities


Header:
  - Summary: X new grants, Y urgent (< 7 days)
  - Quick stats: Avg match score, top funder types


Body (per grant):
  - Title (linked to URL)
  - Funder name and type
  - Match score (color-coded: >0.9 green, 0.7-0.9 yellow)
  - LLM reasoning (2-3 sentence explanation)
  - Deadline (bolded if < 7 days)
  - Funding range
  - Quick actions: 'Mark as Applied', 'Not Relevant', 'Snooze'


Footer:
  - Link to full dashboard
  - Feedback buttons (thumbs up/down)
```

8.2 Email Delivery Implementation

```
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
```

```
def send_daily_digest(recipients, grants):
    msg = MIMEMultipart('alternative')
    msg['Subject'] = f'Daily Grant Digest - {len(grants)} Opportunities'
    msg['From'] = 'grants@techonhand.org'
    msg['To'] = ', '.join(recipients)

    html_content = render_digest_template(grants)
    msg.attach(MIMEText(html_content, 'html'))

    with smtplib.SMTP('smtp.gmail.com', 587) as server:
        server.starttls()
        server.login(SMTP_USER, SMTP_PASSWORD)
        server.send_message(msg)
```

8.3 Future: Slack Integration (Optional)

For teams that prefer collaborative tools, Slack integration can be added in Phase 2.

```
import slack_sdk


client = slack_sdk.WebClient(token=SLACK_BOT_TOKEN)


client.chat_postMessage(
    channel='#grants',
    blocks=[{
        'type': 'section',
        'text': {
            'type': 'mrkdwn',
            'text': f'*{grant.title}*Match: {grant.final_score:.0%}'
        }
    }]
)
```

9. Admin Dashboard (FastAPI)

9.1 Dashboard Features

- Source monitoring: Status, last checked, success rate
- Recent grants: Browse, filter, manual match override
- Match history: View past digests, feedback tracking
- System health: Database size, embedding generation queue
- Configuration: Update mission statement, adjust score thresholds

9.2 FastAPI Implementation

```
from fastapi import FastAPI, Depends
from fastapi.responses import HTMLResponse
import psycopg2

app = FastAPI()

@app.get('/dashboard', response_class=HTMLResponse)
async def dashboard():
    conn = get_db_connection()

    sources = conn.execute('SELECT * FROM sources').fetchall()
    recent_grants = conn.execute('''
        SELECT g.*, m.final_score, m.llm_reasoning
        FROM grants g
        LEFT JOIN grant_matches m ON g.id = m.grant_id
        ORDER BY g.discovered_at DESC
        LIMIT 50
    ''').fetchall()

    return render_template('dashboard.html',
                           sources=sources,
                           grants=recent_grants)

@app.post('/sources/{source_id}/toggle')
async def toggle_source(source_id: str):
    conn.execute('UPDATE sources SET is_active = NOT is_active WHERE id = %s',
                 (source_id,))
```

```
    return {'status': 'success'}
```

10. Configuration Management

10.1 Environment Variables

```
# .env
DATABASE_URL=postgresql://user:password@localhost:5432/grants_db
REDIS_URL=redis://localhost:6379/0

# Email
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=grants@techonhand.org
SMTP_PASSWORD=app_specific_password

# Models
EMBEDDING_MODEL=all-MiniLM-L6-v2
RERANK_MODEL=cross-encoder/ms-marco-MiniLM-L6-v2
LLM_MODEL=llama3.1:8b

# Thresholds
RETRIEVAL_THRESHOLD=0.5
RERANK_TOP_K=20
LLM_THRESHOLD=0.7
DIGEST_MIN_SCORE=0.7

# Admin
ADMIN_USERNAME=admin
ADMIN_PASSWORD_HASH=bcrypt_hash_here
```

10.2 Sources Configuration (YAML)

```
# config/sources.yaml
sources:
  - name: DevelopmentAid
    type: rss
    url: https://www.developmentaid.org/rss/grants
    check_interval_hours: 4
    is_active: true
```

```yaml
  - name: FundsforNGOs Africa
    type: rss
    url: https://www2.fundsforngos.org/category/africa/feed/
    check_interval_hours: 6
    is_active: true

  - name: Terra Viva Grants
    type: rss
    url: https://terra1830.rssing.com/index.php
    check_interval_hours: 12
    is_active: true
```

10.3 Mission Configuration

```yaml
# config/mission.yaml
organization:
  name: Tech On Hand
  mission: |
    Tech On Hand bridges the digital divide in Kenya by refurbishing donated
    computer hardware and distributing it to schools and youth centers...

  focus_areas:
    - Digital literacy
    - Hardware refurbishment
    - E-waste management
    - Youth skills development
    - Educational technology

  geographic_focus:
    - Kenya
    - East Africa

  funding_preferences:
    min_amount: 5000
    max_amount: 100000
    currency: USD
```

11. Deployment Guide

11.1 VPS Setup (DigitalOcean Example)

11.1.1 Create Droplet

- OS: Ubuntu 22.04 LTS
- Plan: Basic - 4 GB RAM / 2 vCPUs ($24/month)
- Datacenter: Frankfurt (lowest latency to Nairobi)
- SSH Key: Add your public key

11.1.2 Initial Server Configuration

```
# SSH into server
ssh root@your_server_ip

# Update system
apt update && apt upgrade -y

# Create non-root user
adduser grantbot
usermod -aG sudo grantbot
su - grantbot
```

11.2 Install Dependencies

```
# Install Python 3.11
sudo apt install python3.11 python3.11-venv python3-pip -y

# Install PostgreSQL
sudo apt install postgresql postgresql-contrib -y

# Install Redis
sudo apt install redis-server -y

# Install Ollama (for LLM)
curl https://ollama.ai/install.sh | sh

# Install Node.js (for potential scrapers)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

```
sudo apt install nodejs -y
```

11.3 Database Setup

```
# Create database and user
sudo -u postgres psql

CREATE DATABASE grants_db;
CREATE USER grantbot WITH PASSWORD 'secure_password';
GRANT ALL PRIVILEGES ON DATABASE grants_db TO grantbot;
\c grants_db
CREATE EXTENSION vector;  -- pgvector extension
\q
```

11.4 Application Deployment

```
# Clone repository
git clone https://github.com/your-org/grant-intelligence.git
cd grant-intelligence

# Create virtual environment
python3.11 -m venv venv
source venv/bin/activate

# Install Python dependencies
pip install -r requirements.txt

# Download ML models
python scripts/download_models.py

# Download LLM
ollama pull llama3.1:8b

# Setup environment
cp .env.example .env
nano .env  # Edit with your credentials

# Initialize database
```

```
python scripts/init_db.py

# Seed sources
python scripts/seed_sources.py
```

11.5 Process Management (Systemd)

```
# Create systemd service
sudo nano /etc/systemd/system/grantbot.service

[Unit]
Description=Grant Intelligence System
After=network.target postgresql.service redis.service

[Service]
Type=simple
User=grantbot
WorkingDirectory=/home/grantbot/grant-intelligence
Environment="PATH=/home/grantbot/grant-intelligence/venv/bin"
ExecStart=/home/grantbot/grant-intelligence/venv/bin/python main.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target

# Enable and start
sudo systemctl enable grantbot
sudo systemctl start grantbot
sudo systemctl status grantbot
```

11.6 Nginx Reverse Proxy (Optional)

```
# Install Nginx
sudo apt install nginx -y

# Configure reverse proxy
sudo nano /etc/nginx/sites-available/grantbot
```

```
server {
    listen 80;
    server_name grants.techonhand.org;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

# Enable site
sudo ln -s /etc/nginx/sites-available/grantbot /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

12. Security & Compliance

12.1 Kenya Data Protection Act (DPA) Compliance

12.1.1 Data Protection Principles

Principle	Implementation
Purpose Limitation	Only collect grant data for funding discovery purposes
Data Minimization	Do not scrape personal contact details unless essential
Accuracy	Validate all scraped data with Pydantic schemas
Storage Limitation	Purge grants older than 12 months automatically
Security	Encrypt database backups, use secure connections

12.1.2 Personal Data Handling

- Grant officer contacts: Only store if publicly listed on grant page
- Email addresses: Hash before storing for deduplication
- User data: Encrypt passwords with bcrypt, use JWT for sessions

12.2 Ethical Scraping Compliance

12.2.1 robots.txt Compliance

```
import urllib.robotparser


def can_scrape(url):
    rp = urllib.robotparser.RobotFileParser()
    rp.set_url(f'{url}/robots.txt')
    rp.read()
    return rp.can_fetch('TechOnHand-GrantBot', url)
```

12.2.2 Rate Limiting

```
import time
import random


# Implement exponential backoff
```

```
def fetch_with_backoff(url, max_retries=3):
    for attempt in range(max_retries):
        response = requests.get(url)

        if response.status_code == 200:
            return response

        if response.status_code == 429:  # Too Many Requests
            wait_time = 2 ** attempt + random.uniform(0, 1)
            time.sleep(wait_time)
        else:
            break

    return None
```

12.3 Security Hardening

- Firewall: Only allow ports 22 (SSH), 80 (HTTP), 443 (HTTPS)
- SSH: Disable password auth, use key-based authentication only
- Database: Require SSL connections, restrict to localhost
- Secrets: Use environment variables, never commit to Git
- Updates: Enable automatic security updates

```
# Enable UFW firewall
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

13. Monitoring & Operational Excellence

13.1 Health Checks

13.1.1 Application Health Endpoint

```
@app.get('/health')
async def health_check():
    checks = {
        'database': check_database_connection(),
        'redis': check_redis_connection(),
        'ollama': check_ollama_service(),
        'disk_space': check_disk_space()
    }

    all_healthy = all(checks.values())
    status_code = 200 if all_healthy else 503

    return JSONResponse(
        content={'status': 'healthy' if all_healthy else 'degraded',
                 'checks': checks},
        status_code=status_code
    )
```

13.2 Logging Strategy

13.2.1 Log Levels

Level	Use Case	Example
DEBUG	Development only	Embedding vector values
INFO	Normal operations	RSS feed checked, 5 new grants found
WARNING	Degraded but working	Source returned 0 grants (unusual)
ERROR	Recoverable failure	Failed to parse grant, skipping
CRITICAL	Service down	Database unreachable, cannot process grants

13.2.2 Structured Logging

```
import structlog


log = structlog.get_logger()


log.info('rss_feed_processed',
         source='DevelopmentAid',
         grants_found=12,
         grants_new=3,
         duration_seconds=2.4)
```

13.3 Error Alerting

Send alerts to admin email for critical failures.

```
def send_alert(subject, message):
    send_email(
        to=ADMIN_EMAIL,
        subject=f'[ALERT] {subject}',
        body=message
    )


# Example usage
if consecutive_failures > 3:
    send_alert(
        'Source Consistently Failing',
        f'Source {source.name} has failed 3 times in a row.'
    )
```

13.4 Performance Metrics

Metric	Target	Monitoring Method
RSS poll time	< 5 seconds	Log duration per source
Embedding generation	< 100ms per grant	Timer in processing pipeline
Stage 1 retrieval	< 10ms	Database query time
Stage 2 reranking	< 5 seconds for 50 items	Model inference time

Stage 3 LLM validation	< 60 seconds for 20 items	Ollama response time
Daily digest generation	< 2 minutes	End-to-end timer

13.5 Automated Backups

```
# Backup script (cron daily at 2 AM)
#!/bin/bash

BACKUP_DIR=/home/grantbot/backups
DATE=$(date +%Y%m%d_%H%M%S)

# Backup PostgreSQL
pg_dump grants_db | gzip > $BACKUP_DIR/grants_db_$DATE.sql.gz

# Backup configuration
tar -czf $BACKUP_DIR/config_$DATE.tar.gz
/home/grantbot/grant-intelligence/config

# Keep only last 7 days
find $BACKUP_DIR -name '*.gz' -mtime +7 -delete

# Add to crontab
0 2 * * * /home/grantbot/scripts/backup.sh
```

14. Testing Strategy

14.1 Unit Tests

```
# tests/test_deduplication.py
import pytest
from src.processing import deduplicate_grants

def test_url_hash_deduplication():
    grant1 = create_test_grant(url='https://example.com/grant1')
    grant2 = create_test_grant(url='https://example.com/grant1')

    result = deduplicate_grants([grant1, grant2])
    assert len(result) == 1

def test_semantic_deduplication():
    grant1 = create_test_grant(
        title='Digital Literacy Training for Youth',
        description='Computer skills for young people'
    )
    grant2 = create_test_grant(
        title='Youth Computer Education Program',
        description='Teaching young adults digital skills'
    )

    similarity = calculate_semantic_similarity(grant1, grant2)
    assert similarity > 0.9  # Should be detected as duplicates
```

14.2 Integration Tests

```
# tests/test_matching_pipeline.py
import pytest
from src.matching import MatchingPipeline

@pytest.mark.integration
def test_full_matching_pipeline():
    pipeline = MatchingPipeline()
```

```
    # Create test grant
    grant = create_test_grant(
        title='Grant for Refurbished Computer Distribution in Kenya',
        description='Support schools with donated laptops...'
    )

    # Run through all stages
    result = pipeline.process(grant)

    assert result.final_score > 0.7  # Should be high match
    assert 'Kenya' in result.llm_reasoning
    assert 'refurbished' in result.llm_reasoning.lower()
```

14.3 End-to-End Tests

```
# tests/test_e2e.py
@pytest.mark.e2e
def test_rss_to_digest_flow():
    # 1. Poll RSS feed
    poller = RSSPoller()
    grants = poller.poll_all_sources()
    assert len(grants) > 0

    # 2. Process grants
    processor = GrantProcessor()
    processed = processor.process_batch(grants)

    # 3. Match grants
    matcher = MatchingPipeline()
    matches = matcher.match_batch(processed)

    # 4. Generate digest
    digest = DigestGenerator()
    email_content = digest.generate(matches)

    assert 'Daily Grant Digest' in email_content
    assert len(matches) <= 10  # Digest limit
```

15. Phased Development Roadmap

15.1 Phase 1: Foundation (Weeks 1-3)

Week 1: Infrastructure

- Set up VPS, install PostgreSQL, Redis, Ollama
- Create database schema and migrations
- Implement configuration management
- Set up Git repository and CI/CD

Week 2: RSS Ingestion

- Build RSS poller for 3 sources
- Implement data normalization with Pydantic
- Create deduplication logic (URL hash + title)
- Set up scheduled polling (APScheduler)

Week 3: Basic Matching

- Download and test embedding models
- Implement Stage 1 retrieval (bi-encoder)
- Create simple email digest template
- Test end-to-end flow with sample data

Deliverable: Working system that finds grants and sends daily emails with basic relevance scores.

15.2 Phase 2: Advanced Matching (Weeks 4-6)

Week 4: Multi-Stage Ranking

- Implement Stage 2 cross-encoder reranking
- Set up Ollama with Llama 3.1 8B
- Create LLM validation prompts and parsing
- Integrate weighted final score calculation

Week 5: Refinement

- Add semantic deduplication (embeddings)
- Implement geographic eligibility extraction
- Create cause tag assignment logic
- Build feedback loop (thumbs up/down in emails)

Week 6: Admin Dashboard

- Build FastAPI dashboard
- Create source monitoring interface
- Implement manual match override
- Add system health metrics

Deliverable: Production-ready system with sophisticated matching and admin controls.

15.3 Phase 3: Expansion (Weeks 7-10)

Week 7-8: Kenyan Sources

- Build scrapers for NRF Kenya, YEDF, ICT Authority
- Implement Playwright for dynamic content
- Test and validate new sources

Week 9: Optimization

- Fine-tune score thresholds based on feedback
- Optimize database queries and indexes
- Implement caching for embeddings
- Set up automated backups

Week 10: Documentation & Training

- Write user documentation
- Create video tutorials for admin dashboard
- Train grant writers on system usage
- Conduct system handover

Deliverable: Fully operational system with local and international sources, optimized and documented.

15.4 Future Enhancements (Beyond Week 10)

- Slack integration for real-time alerts
- WhatsApp notifications for urgent deadlines
- CRM integration (Instrumentl, Good Grants)
- Automated application tracking
- Grant writing assistant (LLM-powered)
- Historical success analysis (which funders awarded grants)
- Collaborative filtering (learn from other NGOs' successes)

16. Total Cost of Ownership (TCO)

16.1 Monthly Infrastructure Costs

Item	Cost (USD)	Notes
VPS (4GB RAM)	$24-48	DigitalOcean or AWS t3.medium
Domain name	$1	grants.techonhand.org
SSL certificate	$0	Let's Encrypt (free)
Email service	$0	Gmail SMTP (free tier)
Total Infrastructure	$25-49	Within budget

16.2 One-Time Development Costs

Item	Hours	Rate	Cost (USD)
Phase 1 (3 weeks)	120	$50	$6,000
Phase 2 (3 weeks)	120	$50	$6,000
Phase 3 (4 weeks)	160	$50	$8,000
Total Development	400		$20,000

Note: Development costs can be reduced by:

- Using volunteer developers from tech community
- Applying for tech-for-good grants (Anthropic, Google.org)
- University partnerships (capstone projects)
- Open-sourcing for community contributions

16.3 Annual Operating Costs

Item	Annual Cost (USD)
Infrastructure	$300-600
Maintenance (10 hours/month)	$6,000
Domain renewal	$12
Backup storage	$60
Total Annual	$6,372-6,672

16.4 Return on Investment (ROI)

Metric	Before	After	Improvement
Hours spent searching/week	20	5	75% reduction
Grants discovered/month	10	30	3x increase
Grant success rate	5%	15%	3x improvement
Annual grants won	6	18	12 additional grants

If each grant averages $25,000, winning 12 additional grants/year = $300,000 additional funding. ROI: 4,400% in first year.

17. Conclusion & Next Steps

17.1 Summary

This technical specification provides a complete blueprint for building an automated grant intelligence system optimized for technology-focused NGOs in Kenya. The system leverages modern NLP and semantic search to transform grant discovery from a manual, labor-intensive process into an intelligent, automated pipeline that surfaces only the most relevant opportunities.

17.2 Key Advantages

- Cost-effective: $25-50/month, all open-source stack
- Accurate: Multi-stage ranking prioritizes precision over recall
- Scalable: Can add new sources and increase processing capacity
- Maintainable: Strong technical team can modify and extend
- Compliant: Respects Kenya DPA and ethical scraping standards

17.3 Immediate Next Steps

Step	Owner	Deadline
Review and approve specification	Leadership	Week 1
Secure funding for development	Finance	Week 2
Hire/assign developers	HR/Tech Lead	Week 3
Set up development environment	Tech Team	Week 4
Begin Phase 1 implementation	Tech Team	Week 5

17.4 Success Criteria (3 Months)

- System operational with 3 RSS sources monitored
- Daily digests delivered to grant writers
- 80%+ precision on surfaced opportunities
- 15+ hours/week time savings measured
- At least 2 grant applications submitted from system-discovered opportunities

17.5 Long-Term Vision

This system represents the first step in building a comprehensive grants management platform. Future iterations could expand to include:

- Automated grant writing assistance using LLMs
- Collaborative learning from other NGOs' successes
- Predictive modeling of grant success probability
- Integration with donor management systems
- Open-source release to benefit the wider NGO ecosystem

By investing in this infrastructure now, Tech On Hand positions itself as a data-driven, technologically sophisticated organization that can compete effectively in the increasingly competitive landscape of international development funding.

Appendix A: Sample Code Repository Structure

```
grant-intelligence/
├── config/
│   ├── sources.yaml
│   ├── mission.yaml
│   └── .env.example
├── src/
│   ├── ingestion/
│   │   ├── rss_poller.py
│   │   └── scrapers/
│   ├── processing/
│   │   ├── normalizer.py
│   │   ├── deduplicator.py
│   │   └── enricher.py
│   ├── matching/
│   │   ├── pipeline.py
│   │   ├── retrieval.py
│   │   ├── reranking.py
│   │   └── llm_validation.py
│   ├── delivery/
│   │   ├── digest_generator.py
│   │   └── email_sender.py
│   └── admin/
│       └── dashboard.py
├── scripts/
│   ├── init_db.py
│   ├── download_models.py
│   ├── seed_sources.py
│   └── backup.sh
├── tests/
│   ├── test_deduplication.py
│   ├── test_matching.py
│   └── test_e2e.py
├── models/  # Downloaded ML models
├── logs/
├── requirements.txt
```

```
├── main.py
└── README.md
```

Appendix B: Key Dependencies (requirements.txt)

```
# Core
fastapi==0.109.0
uvicorn==0.27.0
psycopg2-binary==2.9.9
redis==5.0.1
pydantic==2.5.3

# Scheduling
APScheduler==3.10.4

# HTTP & Scraping
httpx==0.26.0
feedparser==6.0.11
scrapy==2.11.0
playwright==1.41.0

# ML & NLP
sentence-transformers==2.3.1
torch==2.1.2

# Database
pgvector==0.2.4
sqlalchemy==2.0.25

# LLM
ollama==0.1.20

# Utilities
python-dotenv==1.0.0
pyyaml==6.0.1
structlog==24.1.0
pytest==7.4.4
```

Appendix C: Contact & Support

For questions or technical support during implementation:

- Technical Lead: [Your Name]
- Email: tech@techonhand.org
- Documentation: https://github.com/your-org/grant-intelligence/wiki
- Issue Tracker: https://github.com/your-org/grant-intelligence/issues

Document Version: 1.0

Last Updated: February 10, 2026

Author: Technical Architecture Team

Status: Ready for Implementation